

Program-Level Proof Rules for Almost-Sure Termination of Probabilistic Sampling Algorithms

ANONYMOUS AUTHOR(S)

Almost-sure termination (AST) is indispensable for turning partial-correctness arguments about randomized algorithms into total-correctness guarantees. Recent work provides a sound and relatively complete AST rule for probabilistic control-flow graphs (pCFGs), but applying it to source programs requires a translation and location-dependent certificates. We present a new proof rule directly at the program level. The rule interprets a loop body as a distribution transformer and separates two obligations: a non-negative supermartingale controls escape to unbounded regions, while a natural-valued variant decreases with uniformly positive probability and is bounded on every supermartingale sublevel set. Consequently, certificates describe complete loop iterations rather than individual control-flow edges. We prove the rule sound and relatively complete for probabilistic while-loops with loop-free bodies by relating it to the pCFG rule.

We first use a one-dimensional random walk to expose the rule’s core mechanism. We then compare pCFG-level and program-level proofs for the Fast Dice Roller, showing that both approaches apply, while the program-level proof reuses an existing distributional invariant and avoids location-indexed certificate engineering. Finally, we establish AST of the Fast Loaded Dice Roller and the Han–Hoshi sampler. These two case studies particularly highlight the practical advantages of the program-level rule: rejection and restart in the Fast Loaded Dice Roller are handled by a compact argument over complete tree-traversal iterations, while the dyadic refinement performed by Han–Hoshi admits an intuitive certificate based on the target-distribution boundaries contained in the current interval. In both cases, the program-level proofs are shorter and more direct than corresponding pCFG proofs, which would require certificates for every intermediate control location. Together with existing partial-correctness results, our AST proofs yield total-correctness guarantees for all three samplers.

1 Introduction

Randomized algorithms are routinely used in simulation, machine learning, network protocols, and exact sampling. Their correctness has two independent dimensions. *Partial correctness* states that, whenever an algorithm returns, its result has the intended distribution. *Almost-sure termination* (AST) states that it returns with probability one. Only their combination establishes total correctness: without AST, a sampler may produce the right distribution on terminating runs while losing probability mass to divergence.

AST proofs are subtle. A probabilistic loop may terminate almost surely even though its expected runtime is infinite and even though no globally bounded ranking function exists. The symmetric random walk on the non-negative integers is the canonical example: from $x > 0$, move to $x - 1$ or $x + 1$ with equal probability and stop at zero. It reaches zero with probability one, yet its expected hitting time is infinite. Classical finite-state or bounded-variant arguments therefore do not suffice.

Majumdar and Sathiyararayanan recently introduced a sound and relatively complete rule for probabilistic termination at the level of probabilistic control-flow graphs (pCFGs) [28]. Their rule combines a supermartingale with a natural-valued variant. The supermartingale controls the probability of escaping to regions in which the variant becomes large; within each of its sublevel sets, the variant is bounded and decreases with a uniformly positive probability. This is a strong semantic characterization of AST.

The pCFG level, however, is not always the right abstraction for proving source programs. A loop iteration that is conceptually a single sampling step becomes a collection of locations and transitions. Certificates must then be defined at every location and adjusted around assignments, guards, and probabilistic branches. For the Fast Dice Roller (FDR), for example, a simple source-level observation about the growth of $v - c$ turns into a location-indexed variant with several cases. This

syntactic overhead is especially unfortunate for exact samplers, whose functional correctness is already naturally expressed by *distributional invariants* and distribution-transformer semantics [41].

Our approach. We lift the pCFG insight to the source-program level. Rather than asking a certificate to decrease along every internal control-flow edge, our rule summarizes the complete effect of one loop-body execution by its post-distribution. Let $C = \text{while } (b) \{B\}$, where B is loop-free. The rule requires:

- (i) a distributional invariant I closed under one loop iteration;
- (ii) a non-negative supermartingale v whose expected value does not increase after executing B ;
- (iii) a natural-valued variant u that, with probability at least $\varepsilon > 0$, is smaller after executing B ; and
- (iv) boundedness of u in every sublevel set of v .

All obligations are stated over source states and the distribution transformer $\text{sp}[[B]]$. The rule therefore reasons at the semantic granularity at which programmers understand a probabilistic loop: one iteration at a time.

Why sampling algorithms? Exact sampling algorithms provide a natural setting in which to evaluate AST proof rules. Their loop invariants often describe entire distributions rather than merely state predicates; rejection sampling introduces restarts that defeat naïve pointwise descent arguments; and total correctness requires the AST proof to compose with a pre-existing output-distribution proof. Our formulation directly supports this workflow. Reachability information is extracted from the support of a distributional invariant, while the richer equal-mass constraints used for partial correctness can be reused unchanged.

We study three samplers that separate general applicability from practical advantage. First, the Fast Dice Roller (FDR) [27] samples uniformly from $\{0, \dots, n-1\}$ using fair bits. Both the pCFG rule and our program-level rule apply. This direct comparison shows that the new rule does not merely prove a different class of examples: it gives a markedly shorter and more modular proof of the same algorithm. Second, the Fast Loaded Dice Roller (FLDR) [33] samples from an arbitrary finite discrete distribution using a compact decision-tree representation and rejection. At program level, its key termination argument is captured over complete tree-traversal iterations, avoiding certificates for the intermediate control locations introduced by a pCFG translation. Third, the Han-Hoshi sampler repeatedly refines a dyadic interval until it lies entirely within one of the target-distribution intervals. Its AST proof reduces to the intuitive observation that bisecting the current interval distributes its internal target boundaries over the two children, at least one of which contains strictly fewer boundaries. For both FLDR and Han-Hoshi, the program-level rule exposes the essential probabilistic argument directly, resulting in proofs that are shorter and more intuitive than corresponding location-by-location pCFG proofs. Together, these examples highlight the practical advantages of reasoning about complete loop iterations.

Contributions. This paper makes the following contributions.

- (1) We present a new program-level proof rule for AST of probabilistic while-loops with loop-free bodies. The rule is phrased entirely in distribution-transformer semantics and cleanly separates the supermartingale and variant obligations (Section 4).
- (2) We establish soundness by compiling program-level certificates into certificates for the relatively complete pCFG rule. Conversely, we derive program-level certificates from pCFG certificates, yielding relative completeness with respect to the underlying first-order arithmetic theory (Section 5).

- (3) We illustrate the mechanism on the one-dimensional random walk and explain how the source-level rule removes location-dependent bookkeeping (Section 6).
- (4) We give parallel pCFG- and program-level AST proofs for FDR. The comparison isolates the reduction in proof complexity and shows how to reuse an existing distributional invariant (Section 7.1).
- (5) We prove AST of FLDR and Han–Hoshi at program level. FLDR captures rejection and restart through an iteration-level argument over its decision tree, while Han–Hoshi uses a compact certificate based on the target-distribution boundaries contained in the current dyadic interval. These case studies demonstrate how the program-level rule yields shorter and more intuitive proofs by avoiding location-dependent certificate engineering. Combined with existing partial-correctness results, our AST arguments provide total-correctness proofs for all three samplers (Sections 7.2 and 7.3).

Scope. The present rule targets a single while-loop with loop-free body. This restriction makes “one iteration” a finite distribution transformer step and keeps the connection to pCFG paths transparent. It already covers the main sampling loops studied here. Nested loops can be handled modularly once AST summaries for inner loops are available; a fully compositional extension is left for future work.

Outline. Section 2 introduces probabilistic programs, subdistributions, AST, and distributional invariants. Section 3 recalls the pCFG rule that underpins our metatheory. Section 4 states the new rule and gives intuition. Section 5 proves soundness and relative completeness. The random-walk example appears in Section 6. The FDR comparison and the FLDR and Han–Hoshi case studies appear in Sections 7.1 to 7.3, respectively. We discuss related work in Section 8 and conclude in Section 9.

2 Preliminaries

We fix a finite set Var of program variables and a countable state space Σ ; a state $\sigma \in \Sigma$ maps variables to rational values. Arithmetic and Boolean expressions are interpreted as functions on states. For a Boolean expression b , let $\llbracket b \rrbracket = \{\sigma \mid \sigma \models b\}$ and let $[b]$ denote its Iverson bracket. Updating x in σ to $a(\sigma)$ is written $\sigma[x/a(\sigma)]$.

2.1 Probabilistic programs and subdistributions

We consider the probabilistic guarded-command language

$$C ::= \text{skip} \mid x := a \mid C ; C \mid \{C\} [p] \{C\} \mid \text{if } (b) \{C\} \text{ else } \{C\} \mid \text{while } (b) \{C\},$$

where $p \in [0, 1] \cap \mathbb{Q}$. A *subdistribution* is a function $\mu : \Sigma \rightarrow \mathbb{R}_{\geq 0}$ of weight $|\mu| = \sum_{\sigma} \mu(\sigma) \leq 1$. We write $\Delta_{\leq 1} \Sigma$ for all subdistributions, $\text{supp}(\mu) = \{\sigma \mid \mu(\sigma) > 0\}$ for the support, and δ_{σ} for the Dirac distribution at σ . Missing mass represents nontermination.

Programs denote linear post-distribution transformers $\text{sp}[\llbracket C \rrbracket]$. The clauses needed below are

$$\begin{aligned} \text{sp}[\llbracket \text{skip} \rrbracket](\mu) &= \mu, \\ \text{sp}[\llbracket x := a \rrbracket](\mu)(\sigma) &= \sum_{\sigma' = \sigma[x/a(\sigma')]} \mu(\sigma'), \\ \text{sp}[\llbracket \{C_1\} [p] \{C_2\} \rrbracket](\mu) &= \text{sp}[\llbracket C_1 \rrbracket](p\mu) + \text{sp}[\llbracket C_2 \rrbracket]((1-p)\mu), \\ \text{sp}[\llbracket C_1 ; C_2 \rrbracket](\mu) &= \text{sp}[\llbracket C_2 \rrbracket](\text{sp}[\llbracket C_1 \rrbracket](\mu)), \\ \text{sp}[\llbracket \text{if } (b) \{C_1\} \text{ else } \{C_2\} \rrbracket](\mu) &= \text{sp}[\llbracket C_1 \rrbracket](\llbracket b \rrbracket \mu) + \text{sp}[\llbracket C_2 \rrbracket](\llbracket \neg b \rrbracket \mu). \end{aligned}$$

For a loop $C = \text{while } (b) \{B\}$, let $F_\mu(e) = \mu + \text{sp}[B]([b]e)$. Its semantics is

$$\text{sp}[C](\mu) = [\neg b] \cdot \text{lfp } F_\mu.$$

Equivalently, the Kleene approximants collect executions that leave the loop after finitely many iterations. Linearity gives $\text{sp}[C](\sum_i p_i \delta_{\sigma_i}) = \sum_i p_i \text{sp}[C](\delta_{\sigma_i})$, which lets us reduce one-iteration obligations to Dirac inputs.

Definition 2.1 (Almost-sure termination). A program C is *almost-surely terminating* on $M \subseteq \Delta_{\leq 1}\Sigma$ if $|\text{sp}[C](\mu)| = |\mu|$ for every $\mu \in M$.

Thus AST means that execution loses no probability mass. We do not require positive AST: the expected runtime may be infinite.

2.2 Distributional invariants

Our assertions are sets of distributions rather than predicates over individual states. This aligns the termination argument with existing partial-correctness proofs for sampling algorithms [41].

Definition 2.2 (Distributional invariant). Let $C = \text{while } (b) \{B\}$ and $M \subseteq \Delta_{\leq 1}\Sigma$. A set $I \subseteq \Delta_{\leq 1}\Sigma$ is a *distributional invariant* for C and M if

$$M \subseteq I \quad \text{and} \quad [\neg b]\mu + \text{sp}[B]([b]\mu) \in I \quad \text{for all } \mu \in I.$$

We abbreviate $\text{supp}(I) = \bigcup_{\mu \in I} \text{supp}(\mu)$.

The invariant can simultaneously carry state-space bounds and genuinely distributional information such as equal probability of states in a row of a decision tree. The AST rule uses the support information, while retaining the stronger invariant permits direct composition with partial correctness.

2.3 Supermartingales, variants, and sublevel sets

A function $v : \text{supp}(I) \rightarrow \mathbb{R}_{\geq 0}$ is a one-iteration supermartingale for loop body B if, for every enabled state σ ,

$$\sum_{\sigma' \in \Sigma} \text{sp}[B](\delta_\sigma)(\sigma') v(\sigma') \leq v(\sigma).$$

For $r \in \mathbb{R}_{\geq 0}$, its sublevel set is $v_{\leq r} = \{\sigma \in \text{supp}(I) \mid v(\sigma) \leq r\}$. A variant $u : \text{supp}(I) \rightarrow \mathbb{N}$ need not decrease in expectation or on every outcome. We only require a uniformly positive chance to move to a smaller value. The role of v is to ensure that executions cannot escape all regions in which u has a finite upper bound.

This division of labour matters for rejection samplers. A rejected run may reset u to a larger value; AST follows because progress remains possible with nonzero probability while v controls the size of the relevant search space.

3 AST Certificates for Probabilistic Control-Flow Graphs

Our program-level rule builds on the sound and relatively complete pCFG rule of Majumdar and Sathiyararayanan [28]. We state their rule in full before relating its location-dependent certificates to our distribution-transformer formulation.

A probabilistic control-flow graph (pCFG) has configurations (ℓ, σ) consisting of a control location ℓ and a program state σ . Its locations are partitioned into assignment locations L_A , conditional locations L_C , and probabilistic locations L_P . Assignment and conditional configurations have a unique enabled successor. At a probabilistic location, an outgoing edge is selected according to its fixed probability. The distinguished terminal location ℓ_{out} is absorbing.

For a configuration $\gamma = (\ell, \sigma)$, let $\text{Succ}(\gamma)$ denote the set of its successor configurations. If $\ell \in L_P$, let $\Pr(\gamma, \gamma')$ denote the probability of moving from γ to $\gamma' \in \text{Succ}(\gamma)$. An invariant Inv is *inductive* for a set E of initial states if

$$\{(\ell_{\text{init}}, \sigma) \mid \sigma \in E\} \subseteq \text{Inv}$$

and Inv is closed under all positive-probability transitions.

The rule uses two certificates. A non-negative supermartingale V controls the probability of reaching configurations arbitrarily far from termination. A natural-valued variant U provides local progress towards termination. For $r \in \mathbb{R}_{\geq 0}$, define the V -sublevel set

$$V_{\leq r} = \{\gamma \in \text{Inv} \mid V(\gamma) \leq r\}.$$

THEOREM 3.1 (PCFG AST RULE, ADAPTED FROM [28]). *Let $\mathcal{G}$ be a pCFG with initial location ℓ_{init} and terminal location ℓ_{out} , and let E be a set of initial states. The pCFG $\mathcal{G}$ is almost-surely terminating from E if there exist the following objects:*

(1) *An inductive invariant Inv containing $(\ell_{\text{init}}, \sigma)$ for every $\sigma \in E$.*

(2) *A function $V : \text{Inv} \rightarrow \mathbb{R}_{\geq 0}$ satisfying:*

(a)

$$V(\ell, \sigma) = 0 \iff \ell = \ell_{\text{out}}.$$

(b) *For every $\gamma = (\ell, \sigma) \in \text{Inv}$ with $\ell \in L_A \cup L_C$ and unique successor $\gamma' \in \text{Succ}(\gamma)$,*

$$V(\gamma') \leq V(\gamma).$$

(c) *For every $\gamma = (\ell, \sigma) \in \text{Inv}$ with $\ell \in L_P$,*

$$\sum_{\gamma' \in \text{Succ}(\gamma)} \Pr(\gamma, \gamma') V(\gamma') \leq V(\gamma).$$

(3) *A function $U : \text{Inv} \rightarrow \mathbb{N}$ satisfying:*

(a)

$$U(\ell, \sigma) = 0 \iff \ell = \ell_{\text{out}}.$$

(b) *For every $\gamma = (\ell, \sigma) \in \text{Inv}$ with $\ell \in L_A \cup L_C$ and unique successor $\gamma' \in \text{Succ}(\gamma)$,*

$$U(\gamma') < U(\gamma).$$

(c) *For every $r \in \mathbb{R}_{\geq 0}$:*

(i) *the set $\{U(\gamma) \mid \gamma \in V_{\leq r}\}$ is bounded; and*

(ii) *there exists $\varepsilon_r > 0$ such that, for every $\gamma = (\ell, \sigma) \in V_{\leq r}$ with $\ell \in L_P$,*

$$\sum_{\substack{\gamma' \in \text{Succ}(\gamma) \\ U(\gamma') < U(\gamma)}} \Pr(\gamma, \gamma') \geq \varepsilon_r.$$

Moreover, the rule is relatively complete with respect to the underlying first-order theory used to express the invariant and the certificates.

The soundness argument has two complementary layers. Fix a sublevel set $V_{\leq r}$. Since U is natural-valued and bounded on this set, it ranges over a finite interval. At deterministic locations it strictly decreases, while at probabilistic locations it decreases with probability at least ε_r . Consequently, a run that remains inside $V_{\leq r}$ cannot avoid termination with positive probability.

It remains to account for runs that leave every fixed sublevel set. The supermartingale conditions ensure that V does not increase in expectation. Standard supermartingale arguments therefore make indefinite escape through increasingly large sublevel sets a probability-zero event. Combining both layers yields almost-sure termination. Unlike proof rules based on a globally bounded variant,

the sublevel-set formulation also handles null-recurrent programs, such as the symmetric one-dimensional random walk.

Although semantically powerful, the pCFG rule is syntactically fine-grained. Suppose that one source-level loop iteration consists of a guard, two assignments, a probabilistic choice, and a conditional. Its pCFG translation introduces a separate control location for each of these operations. A proof must therefore define U and V at every intermediate location. Location-dependent constants, scaling factors, and offsets may be needed solely to ensure the required inequalities along the internal transitions of one source-level iteration.

Our rule retains the same two-layer termination argument but moves it to program level. Instead of assigning certificates to intermediate control locations, it contracts each complete execution of a loop-free body into one distribution-transformer step. The certificates can therefore express progress over complete loop iterations, matching the abstraction at which sampling algorithms are naturally understood.

Translation assumption. The standard structural translation $\mathcal{T}[C]$ maps program syntax to a pCFG. Assignments become update edges, conditionals become guarded edges, and probabilistic choices become probabilistic locations. Sequential composition connects the exit of one component to the entry of the next, while a while-loop connects the body exit back to the guard.

For a loop-free body B , paths through $\mathcal{T}[B]$ from its entry location ℓ_{in}^B to its exit location ℓ_{out}^B correspond exactly to the outcomes of its distribution-transformer semantics:

$$\text{sp}[\![B]\!](\delta_\sigma)(\sigma') = \Pr(\text{paths in } \mathcal{T}[B] \text{ from } (\ell_{\text{in}}^B, \sigma) \text{ to } (\ell_{\text{out}}^B, \sigma')).$$

Consequently, a program C is AST on a set M of initial subdistributions exactly when $\mathcal{T}[C]$ reaches its terminal location almost surely from every state in $\text{supp}(M)$. A structural proof of this correspondence is summarized in Section A. This correspondence provides the bridge used in Section 5 to transfer soundness and relative completeness between the pCFG rule and our program-level rule.

4 A Program-Level Rule for AST

Consider a program $C = \text{while}(b) \{B\}$ whose body B is loop-free. Let M be a set of initial subdistributions. We now state the main contribution.

THEOREM 4.1 (PROGRAM-LEVEL AST RULE). *The loop C is AST on M if there exist:*

- (1) *a distributional invariant I for C and M ;*
- (2) *a function $v : \text{supp}(I) \rightarrow \mathbb{R}_{\geq 0}$ such that*
 - (a) *$v(\sigma) = 0$ iff $\sigma \in \text{supp}(I) \cap \llbracket \neg b \rrbracket$;*
 - (b) *for every $\sigma \in \text{supp}(I) \cap \llbracket b \rrbracket$,*

$$\sum_{\sigma'} \text{sp}[\![B]\!](\delta_\sigma)(\sigma') v(\sigma') \leq v(\sigma);$$

- (3) *a function $u : \text{supp}(I) \rightarrow \mathbb{N}$ such that*
 - (a) *$u(\sigma) = 0$ for every $\sigma \in \text{supp}(I) \cap \llbracket \neg b \rrbracket$;*
 - (b) *there exists $\varepsilon > 0$ for which, for every enabled state $\sigma \in \text{supp}(I) \cap \llbracket b \rrbracket$;*

$$\sum_{\sigma' : u(\sigma') < u(\sigma)} \text{sp}[\![B]\!](\delta_\sigma)(\sigma') \geq \varepsilon; \quad (\text{Decrease})$$

- (c) *for every $r \geq 0$, the set $\{u(\sigma) \mid \sigma \in \text{supp}(I), v(\sigma) \leq r\}$ is bounded.*

The conditions deliberately do not require u to decrease in expectation. It may increase sharply on some outcomes, as happens when a rejection sampler restarts. The rule asks only for a fixed

positive mass of outcomes that reduce u . Nor must u be globally bounded. Its boundedness is local to sublevel sets of v , allowing null-recurrent computations.

Soundness proof intuition. Fix a sublevel set $v_{\leq r}$ and let K_r bound u there. From every nonterminal state in this set, a complete loop iteration has probability at least ε of decreasing u . Hence there is probability at least ε^{K_r} of following enough decreasing iterations to reach zero, provided the run stays in the set. Repeating this argument rules out nontermination confined to $v_{\leq r}$. Meanwhile, the supermartingale property bounds the probability of reaching arbitrarily large values of v ; non-negative supermartingale convergence rules out escaping every finite sublevel set with positive probability. Thus all nonterminating behavior has measure zero.

Iteration-level abstraction. The essential difference to Theorem 3.1 is the granularity of the decrease condition. For a body containing m internal control-flow steps, the pCFG rule needs a value at each intermediate location and a descent-compatible branch at each probabilistic node. The new rule evaluates the final post-distribution once. Assignments, guards, and choices internal to B are hidden behind $\text{sp}[B]$. This has three practical consequences:

- (1) certificates are functions of program states, not location–state pairs;
- (2) an existing distributional invariant can be reused directly; and
- (3) algebraic simplification of $\text{sp}[B](\delta_\sigma)$ exposes exactly the outcomes relevant to progress.

Uniform versus sublevel-dependent progress. The pCFG rule permits an ε_r for each sublevel set. We use a global ε because source programs here contain only fixed rational probabilistic choices, and because both samplers use fair bits. A more general statement can replace ε by ε_r throughout without changing the proof architecture.

Modularity. Initial deterministic code preceding the loop is handled by transforming M into the set of loop-entry distributions. For a sampler with preprocessing, we regard the resulting finite data structure as read-only parameters of the sampling loop. The preprocessing itself terminates by ordinary bounded-loop reasoning. This isolates the probabilistic AST argument to the loop where it is needed.

5 Soundness and Relative Completeness

We connect Theorem 4.1 to the pCFG rule through the translation outlined in Section 3. The proofs also explain why contracting a loop body preserves the two certificates.

5.1 From edge-level to iteration-level reasoning

Let B be loop-free and let $\mathcal{T}[B]$ have entry ℓ_{in} and exit ℓ_{out} . Because B is loop-free, every path from entry to exit is finite and there is a maximum path length L_B .

LEMMA 5.1 (SUPERMARTINGALE CONTRACTION). *Let V satisfy the pCFG supermartingale conditions on $\mathcal{T}[B]$. Then for every reachable (ℓ, σ) ,*

$$V(\ell, \sigma) \geq \sum_{\sigma'} \Pr(\ell, \sigma \rightsquigarrow \ell_{\text{out}}, \sigma') V(\ell_{\text{out}}, \sigma').$$

PROOF. Induct backwards over the acyclic control-flow structure of B . At an assignment or guard location, apply monotonicity along the unique enabled successor. At a probabilistic location, apply the local expectation inequality and the induction hypothesis to every successor. The path probabilities partition according to the first edge. $\square$

LEMMA 5.2 (VARIANT PATH). *Let U satisfy the pCFG variant conditions on $\mathcal{T}[B]$. From every reachable entry configuration there is a positive-probability path to an exit configuration with a smaller U -value.*

PROOF. At deterministic locations choose the unique successor, which lowers U . At probabilistic locations choose a positive-probability successor lowering U . Acyclicity ensures that this construction reaches the body exit after finitely many choices. $\square$

5.2 Soundness

THEOREM 5.3 (SOUNDNESS). *The program-level rule in Theorem 4.1 is sound.*

PROOF SKETCH. Assume certificates I, v, u and translate C to $G = \mathcal{T}[C]$. Let Inv contain all configurations reachable from $\text{supp}(M)$. Distributional inductiveness and the translation correspondence ensure that every state at the loop guard belongs to $\text{supp}(I)$.

We lift v to a pCFG supermartingale V . At the loop guard set $V(\ell_g, \sigma) = v(\sigma) + 1$, and set $V = 0$ at the program exit. At an internal body location, define V as the expected value at the end of the remaining body execution. This satisfies equality at probabilistic locations and along all internal deterministic steps. At the back edge, the source-level supermartingale inequality establishes non-increase. Lemma 5.1 formalizes this construction.

For the variant, reserve enough numerical room for internal control flow. At the guard put $U(\ell_g, \sigma) = L_B u(\sigma) + 1$ and put $U = 0$ at the program exit. Define values backwards inside the body: at a deterministic location use the successor value plus one; at a probabilistic location use one plus the minimum successor value. Thus every deterministic edge decreases U , and some probabilistic successor decreases it.

The source-level decrease condition says that, with probability at least ε , a body execution ends in σ' with $u(\sigma') < u(\sigma)$. Since its path has length at most L_B , scaling by L_B ensures that at least one such path descends through the internal pCFG values and returns to a smaller guard value. Fixed program probabilities provide the required positive lower bound at internal probabilistic locations.

Finally, if $v(\sigma) \leq r$, source-level boundedness bounds $u(\sigma)$; multiplication by L_B and addition of at most L_B internal steps preserves boundedness of U . The lifted certificates satisfy Theorem 3.1, so G reaches its exit almost surely. AST preservation by the translation yields $|\text{sp}[\llbracket C \rrbracket](\mu)| = |\mu|$ for every $\mu \in M$. $\square$

5.3 Relative completeness

THEOREM 5.4 (RELATIVE COMPLETENESS). *For AST loops $C = \text{while } (b) \{B\}$ with loop-free body, the program-level rule is relatively complete with respect to the first-order arithmetic theory in which its assertions and certificates are expressed.*

PROOF SKETCH. Suppose C is AST on M . Its translated pCFG G is AST from $\text{supp}(M)$. Relative completeness of Theorem 3.1 yields an inductive invariant Inv and pCFG certificates V, U .

Project the guard-location invariant to distributions:

$$I = \{\mu \in \Delta_{\leq 1} \Sigma \mid \text{supp}(\mu) \subseteq \{\sigma \mid (\ell_g, \sigma) \in Inv\}\}.$$

Operational/denotational correspondence and inductiveness of Inv make I a distributional invariant. Set $v(\sigma) = V(\ell_g, \sigma)$ and $u(\sigma) = U(\ell_g, \sigma)$. The expected non-increase of v after a complete iteration follows from Lemma 5.1 and the equality between path probabilities and $\text{sp}[\llbracket B \rrbracket](\delta_\sigma)$.

By Lemma 5.2, some body path reaches a lower guard-level U -value. Inside a fixed V -sublevel set, the pCFG rule supplies positive lower bounds for all probabilistic descent choices. A body path has at most L_B edges, so the product of finitely many such bounds is positive; taking the minimum

over the bounded certificate range yields the source-level decrease bound. Restricting the pCFG boundedness condition to guard configurations directly bounds u on every v -sublevel set.

The projected objects are definable whenever the original pCFG certificates and reachability assertions are definable in the underlying theory. They satisfy Theorem 4.1. $\square$

The two translations show that no semantic proving power is lost for the considered program class. The benefit is representational: users state certificates once per source state and reason about complete iterations.

6 The One-Dimensional Random Walk

Consider the symmetric random walk

$$x := k; \quad \text{while } (x > 0) \{ \{x := x - 1\} [1/2] \{x := x + 1\} \}, \quad k \in \mathbb{N}_{>0}.$$

This loop terminates almost surely at $x = 0$, although its expected termination time is infinite. It is therefore a compact illustration of why the supermartingale and variant play different roles.

Let $M = \{\delta_\sigma \mid \sigma(x) = k, k > 0\}$ and $I = \{\mu \in \Delta_{\leq 1} \Sigma \mid \text{supp}(\mu) \subseteq \{\sigma \mid \sigma(x) \geq 0\}\}$. The set I is distributionally inductive. Choose

$$v(\sigma) = u(\sigma) = \max\{\sigma(x), 0\}.$$

For every enabled state,

$$\sum_{\sigma'} \text{sp}[B](\delta_\sigma)(\sigma') v(\sigma') = \frac{1}{2}(x - 1) + \frac{1}{2}(x + 1) = x = v(\sigma),$$

so v is in fact a martingale. With probability $1/2$, the left branch is taken and u decreases by one; hence the decrease obligation holds with $\varepsilon = 1/2$. Finally, on $v_{\leq r}$ we have $u \leq r$. Theorem 4.1 proves AST.

The same mathematical idea can be expressed at pCFG level, but the certificate becomes location-dependent. A loop iteration contains the guard, a probabilistic choice, and an assignment. A pCFG variant therefore uses a scaling factor (three in a direct translation) and different additive offsets at the intermediate locations to force strict descent across every deterministic edge. The supermartingale similarly anticipates the next assignment at branch locations. None of this bookkeeping represents probabilistic insight; it is introduced by the operational granularity. The program rule preserves the natural proof: “distance x is a martingale and decreases with probability one half”

A walled walk with $0 \leq x \leq W$ admits an even older bounded-variant argument [29]. Removing the wall destroys global boundedness but not AST. Sublevel-set boundedness is precisely what recovers the unbounded example and previews the sampler proofs below.

The same construction extends to the standard two-dimensional walk when termination is phrased as hitting an appropriate absorbing boundary and a suitable radial or coordinate certificate is available. We keep the one-dimensional version because it isolates the proof rule without geometric side conditions; the substantial case studies concern sampling algorithms.

7 Case Studies: Exact Sampling from Fair Bits

7.1 Fast Dice Roller: the same proof at two levels

The Fast Dice Roller (FDR) samples uniformly from $\{0, \dots, n - 1\}$ using fair coin flips [27]. We use the following program-level presentation:

```

v := 1; c := 0;
while (v < n) do
  v := 2v;
  c := 2c [1/2] c := 2c + 1;

```

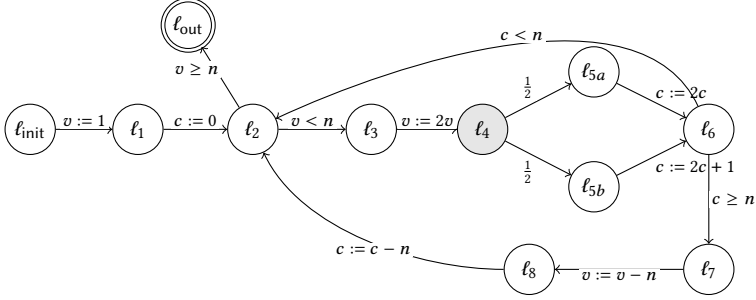


Fig. 1. The pCFG obtained from FDR. The locations $\ell_2, \dots, \ell_8$ expose the loop guard, the update $v := 2v$, the fair-bit choice, the bit-dependent updates of c , and the rejection branch.

```

if (c ≥ n) then v := v - n; c := c - n else skip
end

```

On termination, c is the sampled outcome. Existing distributional-invariant reasoning proves that the terminating output is uniform [41]; AST completes total correctness. The distributional invariant taken from [41] states that at the loop head, the reachable states satisfy

$$0 \leq c < \min(v, n), \quad 0 < v < 2n, \quad n > 0. \quad (1)$$

The distributional invariant additionally states that equal admissible c -values carry equal mass, however only (1) is needed for our termination proof.

Let B_{FDR} denote the body. For a Dirac input δ_σ , its post-distribution is

$$\begin{aligned} \text{sp}[[B_{\text{FDR}}]](\delta_\sigma) = & \frac{1}{2} [c < n] (\delta_{\sigma[v/2v, c/2c]} + \delta_{\sigma[v/2v, c/2c+1]}) \\ & + \frac{1}{2} [c \geq 0] (\delta_{\sigma[v/2v-n, c/2c-n]} + \delta_{\sigma[v/2v-n, c/2c+1-n]}), \end{aligned} \quad (2)$$

where brackets are evaluated after the corresponding updates. This compact expression is the only operational calculation required by the program-level proof.

THEOREM 7.1 (AST OF FDR). *For every $n \in \mathbb{N}_{>0}$, FDR terminates almost surely.*

PROOF. Use the existing distributional invariant and define

$$v(\sigma) = \begin{cases} 0 & v \geq n, \\ n & v < n, \end{cases} \quad u(\sigma) = \begin{cases} 0 & v \geq n, \\ \left\lceil \log_2 \left(\frac{n}{v-c} \right) \right\rceil & v < n. \end{cases}$$

The supermartingale obligation is immediate: n is unchanged and every successor is assigned either n or 0. For progress, inspect the outcome generated by the left fair branch. Before possible subtraction of n , both v and c are doubled; after subtraction their difference remains $2(v-c)$. Hence in either case the denominator of $n/(v-c)$ doubles, so u strictly decreases. The left bit has probability $1/2$, giving $\varepsilon = 1/2$.

For a sublevel set with $r < n$, only terminal states occur. If $r \geq n$, the invariant bounds $n \leq r$, $0 < v < 2n$, and $0 \leq c < v$, so $v-c \geq 1$ and $u \leq \lceil \log_2 r \rceil$. All obligations of Theorem 4.1 hold. $\square$

Comparison with the pCFG proof. The pCFG rule of Majumdar and Sathiyarayanana is applicable to FDR and establishes the same AST statement as Theorem 7.1. Unlike the program-level proof, however, it reasons about individual configurations (ℓ, σ) of the graph in Figure 1. Thus, the initialisation, loop guard, doubling of v , fair-bit choice, conditional, and the two subtraction assignments all become separate proof obligations.

For completeness, we spell out the corresponding pCFG certificate. Write Inv_ℓ for the states admitted at location ℓ . The invariant is obtained by transporting the support information (1) through the intermediate assignments:

$$\begin{aligned}
 Inv_{\ell_{\text{init}}} &:= \{\sigma \mid \sigma(n) \in \mathbb{N}_{>0}\}, \\
 Inv_{\ell_1} &:= \{\sigma \mid \sigma(n) \in \mathbb{N}_{>0} \wedge \sigma(v) = 1\}, \\
 Inv_{\ell_2} &:= \{\sigma \mid \sigma(n) \in \mathbb{N}_{>0} \wedge 1 \leq \sigma(v) < 2\sigma(n) \wedge 0 \leq \sigma(c) < \min\{\sigma(v), \sigma(n)\}\}, \\
 Inv_{\ell_3} &:= \{\sigma \in Inv_{\ell_2} \mid \sigma(v) < \sigma(n)\}, \\
 Inv_{\ell_4} &:= \{\sigma \mid \sigma(n) \in \mathbb{N}_{>0} \wedge 2 \leq \sigma(v) \leq 2\sigma(n) - 2 \wedge 0 \leq \sigma(c) < \frac{\sigma(v)}{2}\}, \\
 Inv_{\ell_{5a}} &:= Inv_{\ell_4}, \quad Inv_{\ell_{5b}} := Inv_{\ell_4}, \\
 Inv_{\ell_6} &:= \{\sigma \mid \sigma(n) \in \mathbb{N}_{>0} \wedge 2 \leq \sigma(v) \leq 2\sigma(n) - 2 \wedge 0 \leq \sigma(c) < \sigma(v)\}, \\
 Inv_{\ell_7} &:= \{\sigma \in Inv_{\ell_6} \mid \sigma(c) \geq \sigma(n)\}, \\
 Inv_{\ell_8} &:= \{\sigma \mid \sigma(n) \in \mathbb{N}_{>0} \wedge 1 \leq \sigma(v) \leq \sigma(n) - 2 \wedge \sigma(n) \leq \sigma(c) < \sigma(v) + \sigma(n)\}, \\
 Inv_{\ell_{\text{out}}} &:= \{\sigma \in Inv_{\ell_2} \mid \sigma(v) \geq \sigma(n)\}.
 \end{aligned}$$

Hence

$$Inv := \bigcup_{\ell \in \{\ell_{\text{init}}, \ell_1, \dots, \ell_8, \ell_{\text{out}}\}} (\{\ell\} \times Inv_\ell)$$

is an inductive pCFG invariant. The loop-head component Inv_{ℓ_2} is precisely the support part of the distributional invariant reused in the program-level proof; the other components are locationspecific consequences of it.

The supermartingale is simple but already locationsensitive through the terminal location:

$$V(\ell, \sigma) := \begin{cases} 0, & \ell = \ell_{\text{out}}, \\ \sigma(n), & \ell \neq \ell_{\text{out}}. \end{cases}$$

Since n is never modified, every nonterminal deterministic successor has the same V -value as its predecessor, while a transition to ℓ_{out} decreases it to 0. At the sole probabilistic location ℓ_4 , both successors retain the value $\sigma(n)$. Thus the supermartingale obligations are immediate. Moreover, every sublevel set $V_{\leq r}$ fixes $n \leq r$; together with the bounds in Inv this makes all non-initial parts of the sublevel set finite.

The pCFG variant must instead account for the exact point within an iteration. One suitable choice is the following locationspecific refinement of the iteration-level distance $\lceil \log_2(n/(v-c)) \rceil$:

$$U(\ell, \sigma) := \begin{cases} 7 \lceil \log_2(\sigma(n)) \rceil + 3, & \ell = \ell_{\text{init}}, \\ 7 \lceil \log_2(\sigma(n)) \rceil + 2, & \ell = \ell_1, \\ 7 \left\lceil \log_2 \left(\frac{\sigma(n)}{\sigma(v) - \sigma(c)} \right) \right\rceil + 1, & \ell = \ell_2, \\ 7 \left\lceil \log_2 \left(\frac{\sigma(n)}{\sigma(v) - \sigma(c)} \right) \right\rceil, & \ell = \ell_3, \\ 7 \left\lceil \log_2 \left(\frac{\sigma(n)}{\frac{\sigma(v)}{2} - \sigma(c)} \right) \right\rceil - 1, & \ell = \ell_4, \\ 7 \left\lceil \log_2 \left(\frac{\sigma(n)}{\sigma(v) - 2\sigma(c)} \right) \right\rceil + 5, & \ell = \ell_{5a}, \\ 7 \left\lceil \log_2 \left(\frac{\sigma(n)}{\sigma(v) - (2\sigma(c) + 1)} \right) \right\rceil + 5, & \ell = \ell_{5b}, \\ 7 \left\lceil \log_2 \left(\frac{\sigma(n)}{\sigma(v) - \sigma(c)} \right) \right\rceil + 4, & \ell = \ell_6, \\ 7 \left\lceil \log_2 \left(\frac{\sigma(n)}{\sigma(v) - \sigma(c)} \right) \right\rceil + 3, & \ell = \ell_7, \\ 7 \left\lceil \log_2 \left(\frac{\sigma(n)}{\sigma(v) + \sigma(n) - \sigma(c)} \right) \right\rceil + 2, & \ell = \ell_8, \\ 0, & \ell = \ell_{\text{out}}. \end{cases}$$

The factor 7 bounds the number of pCFG transitions needed for one source-level loop iteration. The additive offsets ensure strict descent on transitions that merely change the control location. The expressions at $\ell_4, \ell_{5a}, \ell_{5b}$ additionally compensate for the fact that one of the two updates of the iteration has already been executed. For example, at ℓ_4 the current value of v has already been doubled whereas c still has its old value, explaining the denominator $\sigma(v)/2 - \sigma(c)$. At ℓ_{5a} and ℓ_{5b} , in contrast, the two possible updates of c have already been performed. The deterministic decrease conditions can consequently be discharged by checking the edges of Figure 1; the guard edge to ℓ_{out} decreases to 0, and the offsets enforce strict decrease on the remaining deterministic edges. At ℓ_4 , the ℓ_{5a} successor—corresponding to the fair-bit outcome 0—has a strictly smaller variant. This successor has probability $\frac{1}{2}$. Semantically, this is the familiar iteration-level observation that choosing the zero bit doubles $v - c$; the rejection updates subtract n from both variables and hence preserve that difference. In the pCFG proof this single observation is distributed over the locations $\ell_4, \ell_{5a}, \ell_6, \ell_7, \ell_8$.

Finally, U is bounded in every sublevel set $V_{\leq r}$. Indeed, $V \leq r$ implies $\sigma(n) \leq r$, and the invariant bounds all relevant values of v and c ; the two initialization locations are harmless because their variant values depend only on n . Thus the pCFG rule applies with $\varepsilon_r = \frac{1}{2}$ for every r .

In comparison, the program-level proof uses one support invariant, the two state functions displayed in Theorem 7.1, and the single body calculation (2). It also directly reuses the invariant required for the uniform-output proof. Both proof routes have the same proving power, but the program-level rule expresses the algorithmic fact—progress over a complete loop iteration—without introducing locations, offsets, and intermediate state expressions solely for control-flow bookkeeping.

7.2 Fast Loaded Dice Roller: rejection without graph bookkeeping

The Fast Loaded Dice Roller (FLDR) samples outcome i with probability a_i/m , where $a_1, \dots, a_n$ are positive integers and $m = \sum_i a_i$ [33]. Let $k = \lceil \log_2 m \rceil$. Preprocessing adds a rejecting outcome $n+1$ of weight $2^k - m$ and builds a compact representation (H, h) of a binary decision tree. Sampling traverses the tree using fair bits; reaching $n+1$ resets the traversal to the root.

Abstracting the terminating preprocessing, the sampling loop is:

```

while ( $d < \phi_h(c)$ ) do
   $c := c + 1$ ;
   $d := 2d \ [1/2]$   $d := 2d + 1$ ;
  if ( $H[d, c] = n + 1$ ) then  $c := 0$ ;  $d := 0$  else skip
end

```

Here c is the current depth and d the node index; $\phi_h(c)$ identifies the first leaf position in row c . The preprocessing guarantees two facts used below: the tree height is at most k , and no internal node has both children equal to the rejecting leaf. Equivalently, from every enabled state at least one fair-bit successor avoids reset and advances one level towards a valid leaf.

The distributional invariant from the partial-correctness proof bounds reachable states by

$$0 \leq c \leq k, \quad 0 \leq d \leq n, \quad (3)$$

and additionally records equal-mass and output-probability constraints. Again, the latter are retained for total correctness but are not consumed by AST.

THEOREM 7.2 (AST OF FLDR). *For every finite vector of positive integer weights, the FLDR sampling loop terminates almost surely.*

PROOF. Let b be the loop guard. Define

$$v(\sigma) = \begin{cases} 0 & \sigma \not\models b, \\ m & \sigma \models b, \end{cases} \quad u(\sigma) = \begin{cases} 0 & \sigma \not\models b, \\ k - c & \sigma \models b. \end{cases}$$

Neither the weights nor m are modified. Every post-state therefore has v equal to m if the loop remains enabled and 0 otherwise, proving the supermartingale inequality.

Consider an enabled node. Each fair-bit outcome first increments c . If it is not the rejecting leaf, no reset occurs and u decreases by one. The preprocessing property excludes the case in which both children reject. Therefore at least one of the two outcomes has smaller u , and its probability is $1/2$. The decrease condition holds with $\varepsilon = 1/2$ even though the other outcome may reset (c, d) and increase u back to k .

Finally, on $v_{\leq r}$, either $r < m$ and only terminal states occur, or $m \leq r$. In the latter case $k = \lceil \log_2 m \rceil \leq \lceil \log_2 r \rceil$, while (3) gives $0 \leq u = k - c \leq k$. Thus u is bounded in every sublevel set, and Theorem 4.1 applies. $\square$

This proof isolates the actual AST argument in three facts: m is stable, tree height is logarithmically bounded by m , and at least one child avoids rejection. A direct pCFG treatment would additionally require values at the increment, choice, test, and reset locations, plus cases for both children. FLDR therefore illustrates the main contribution more strongly than FDR: the iteration-level rule turns a restart-heavy sampler into a short semantic proof.

From AST to total correctness. The reused distributional invariants establish that the terminating FDR output is uniform and that FLDR returns i with probability a_i/m [41]. Theorems 7.1 and 7.2 show that the corresponding post-distributions have total mass one. Hence the partial-correctness distributions are proper output distributions, yielding total correctness for both algorithms.

7.3 Han–Hoshi Sampling: dyadic refinement without tail bounds

The Han–Hoshi sampler draws from a discrete distribution with positive integer weights $a_1, \dots, a_n$ and total weight $m = \sum_{i=1}^n a_i$. Write $S_i = \sum_{j=1}^i a_j$, with $S_0 = 0$. The algorithm repeatedly refines a dyadic interval until it lies entirely inside one of the target intervals

$$I_i = \left[\frac{S_{i-1}}{m}, \frac{S_i}{m} \right), \quad 1 \leq i \leq n.$$

Its sampling loop is:

```

v := 1; c := 0; r := 0;
while (r = 0) do
  v := 2v;
  c := 2c [ 1 / 2 ] c := 2c + 1;
  r := idx(c, v)
end

```

Here

$$\text{idx}(c, v) := \begin{cases} i, & \text{if } S_{i-1}v \leq cm \text{ and } (c+1)m \leq S_i v \text{ for some } i \in \{1, \dots, n\}, \\ 0, & \text{if no such } i \text{ exists.} \end{cases}$$

The pair (c, v) represents the interval

$$J(c, v) = \left[\frac{c}{v}, \frac{c+1}{v} \right).$$

At loop heads the reachable states satisfy

$$v > 0, \quad 0 \leq c < v, \quad 0 \leq r \leq n. \quad (4)$$

Moreover, after an iteration, $r = 0$ exactly when the selected child interval is not contained in a target interval. As in the preceding case studies, a stronger distributional invariant used for output correctness may be retained; only the support information above is needed for AST.

Let B_{HH} denote the loop body. For $b \in \{0, 1\}$, set

$$c_b = 2c + b, \quad v' = 2v, \quad r_b = \text{idx}(c_b, v').$$

For a Dirac input δ_σ ,

$$\text{sp}[[B_{\text{HH}}]](\delta_\sigma) = \frac{1}{2}\delta_{\sigma[v/v', c/c_0, r/r_0]} + \frac{1}{2}\delta_{\sigma[v/v', c/c_1, r/r_1]}. \quad (5)$$

Thus one complete program-level step is precisely a fair choice between the left and right halves of $J(c, v)$.

THEOREM 7.3 (AST OF HAN–HOSHI). *For every finite vector of positive integer weights, the Han–Hoshi sampling loop terminates almost surely.*

PROOF. For a reachable state σ , let

$$N(\sigma) = \left| \left\{ i \in \{1, \dots, n-1\} \mid \frac{c}{v} < \frac{S_i}{m} < \frac{c+1}{v} \right\} \right|$$

be the number of internal target boundaries lying strictly inside $J(c, v)$. Define the two certificates

$$V(\sigma) = U(\sigma) = \begin{cases} 0, & r \neq 0, \\ \max\{1, N(\sigma)\}, & r = 0. \end{cases}$$

They vanish exactly on terminal states, and U is natural-valued.

Consider an enabled state and write $q = N(\sigma)$. Splitting $J(c, v)$ into its two children cannot create an internal target boundary. Every boundary counted in a child is counted in the parent, while a boundary at the common endpoint of the children is counted in neither. If N_b denotes the number of boundaries strictly inside child b , then

$$N_0 + N_1 \leq q. \quad (6)$$

A child remains enabled exactly when it contains an internal boundary in its interior; hence its post-state certificate is N_b if $N_b > 0$ and zero otherwise. Using (5) and (6), for $q > 0$ we obtain

$$\mathbb{E}_\sigma[V'] = \frac{1}{2}(N_0 + N_1) \leq \frac{q}{2} \leq V(\sigma).$$

If $q = 0$, both children are terminal and the expectation is zero. This proves the supermartingale obligation of Theorem 4.1.

For the decrease obligation, if $q > 0$, at least one of N_0, N_1 is strictly smaller than q : otherwise their sum would be at least $2q$, contradicting (6). The corresponding post-state therefore has strictly smaller U . It is selected with probability $1/2$, so the decrease condition holds with $\varepsilon = 1/2$. If $q = 0$, both children terminate and decrease U from 1 to 0, so the same bound applies.

Finally, $U = V$ on every reachable state. Consequently, on each sublevel set $V \leq R$ we have $U \leq R$. All obligations of Theorem 4.1 hold, and the loop is AST. $\square$

The proof replaces the explicit depth- t tail calculation by one iteration-level fact: bisecting an interval distributes its internal target boundaries over the two children, and at least one child contains strictly fewer. The number of such boundaries serves simultaneously as the supermartingale and the natural-valued variant (with the value 1 for the enabled boundary-free initial case). Hence Han–Hoshi exhibits the same proof pattern as FDR and FLDR and is established directly by the program-level rule of Theorem 4.1.

8 Related Work

Proof rules for almost-sure termination. The classical approach to AST verifies a *variant function* that decreases with a uniformly positive probability, going back to McIver and Morgan [29]; their rule is complete only when the reachable state space is finite. Supermartingale-based rules relax this requirement. McIver et al. [30] give a rule in which a variant, itself required to be a supermartingale, decreases with a state-dependent probability and satisfies an additional progress condition; this rule also handles demonic nondeterminism and extends the class of provably AST programs beyond what bounded-variant arguments allow. Majumdar and Sathiyarayan [28] recently gave the first proof rule for AST that is both sound and relatively complete, formulated over probabilistic control-flow graphs (pCFGs). They further show that the earlier rule of McIver et al. [30] enjoys the same relative-completeness guarantee when read at the program level. Our rule is derived from the pCFG rule of [28] and inherits its soundness and relative completeness, but it is stated directly over distribution-transformer semantics and factors the supermartingale and variant obligations into two independent certificates rather than a single combined one. This factorization is what lets rejection loops reset the variant without disturbing the supermartingale argument, which is central to our Fast Loaded Dice Roller (FLDR) case study. Qualitative AST is the property we target throughout; positive AST, which additionally bounds the expected runtime, is a strictly stronger and differently characterized property that we do not address.

Operational and denotational models. pCFGs and Markov decision processes are the standard operational substrate for termination and reachability analysis of probabilistic and randomized systems [8, 32]. Tools such as Storm [19] and refined analyses of visiting times and stationary behavior [21, 31] operate at this level. Viewing probabilistic systems as *distribution transformers*

rather than as single-state transition systems has recently been fruitful for both Markov chains and MDPs: Aghamov et al. [1, 2] model-check Markov chains through their action on distributions, and Akshay et al. [3, 4] synthesize invariants and certified policies for MDPs under distributional reach-avoidance objectives; distribution-based objectives for MDPs are also studied in [5]. Our rule applies the same distribution-transformer perspective one level higher, directly to program syntax rather than to an underlying automaton, which avoids the location-indexed bookkeeping that a translation to pCFGs or MDPs otherwise imposes.

Denotational semantics and program logics for probabilistic programs. The denotational, post-distribution-transformer semantics we build on originates with Kozen [24, 25] and underlies the weakest-precondition and weakest pre-expectation calculi of McIver and Morgan [29] and Kaminski [22], in the tradition of Dijkstra’s predicate-transformer semantics [14, 15]. Hoare-style program logics for probabilistic programs were introduced by den Hartog and de Vink [13] and extended by the Ellora framework [9]; more recent calculi include quantitative strongest-postcondition reasoning [36], its hyperproperty generalization [37], and modal separation logic for conditional probability [26]. These logics target partial correctness or information flow rather than AST; our rule complements them by supplying the missing termination argument needed to lift a partial-correctness proof to total correctness.

Verifying exact sampling algorithms. Exact discrete samplers built from fair coin flips, following the classical constructions of von Neumann [34] and Lumbroso’s Fast Dice Roller (FDR) [27], have received growing verification attention. Saad et al. introduce the Fast Loaded Dice Roller (FLDR) as a near-optimal sampler for arbitrary discrete distributions [33]; the original paper gives an informal termination argument, while our contribution supplies a proof in the program logic. Bagnall, Stewart, and Banerjee’s Zar compiler [6, 7] formally verifies samplers compiled from probabilistic programs with loops and conditioning in Coq, targeting the random-bit model; generating-function techniques have likewise been used to reason about probabilistic programs and about samplers such as the fast dice roller [12, 18, 23]. Exact and guaranteed-bound inference engines such as PSI [16, 17], the approach of Beutner et al. [11], and scalable exact inference for discrete programs [20] address the complementary problem of computing or bounding the output distribution rather than proving termination. Closest to our work is Zilken and Batz et al.’s framework of distributional invariants [41], which proves partial correctness of FDR-style samplers by tracking distributional invariants under the same post-distribution-transformer semantics we use. That work establishes that FDR’s output is uniformly distributed *conditioned on termination*; our AST proofs for FDR and FLDR are the missing ingredient for total correctness, and our rule is deliberately phrased so that its distributional invariant obligation can reuse the one already constructed for partial correctness.

Nondeterminism. Several lines of work verify probabilistic programs or Markov models in the presence of demonic or angelic nondeterminism [10, 30, 35, 38–40]. Our proof rule targets probabilistic while-loops without nondeterministic choice; extending the distribution-transformer formulation to resolve nondeterminism, e.g., by adapting strategy-based semantics as in [10], is an interesting direction we do not pursue here.

9 Conclusion and Future Work

Achieved results. We presented a new proof rule for almost-sure termination of probabilistic while-loops with loop-free bodies, phrased entirely in distribution-transformer semantics. The rule separates two certificates: a non-negative supermartingale that bounds the probability of escaping to ever-larger sublevel sets, and a natural-valued variant that need only decrease with

a uniformly positive probability and be bounded within each such sublevel set. We proved the rule sound and relatively complete by relating program-level certificates to the pCFG-level rule of Majumdar and Sathiyararayanan [28]: soundness follows by compiling our certificates into pCFG certificates, and relative completeness follows in the converse direction. On the one-dimensional random walk, the rule reproduces the textbook AST argument without any location-indexed bookkeeping. For the Fast Dice Roller, we gave AST proofs both at pCFG level and at program level; the comparison shows that the program-level proof is markedly shorter, reuses an existing distributional invariant, and avoids introducing artificial constants purely to enforce descent across intermediate control-flow edges. Finally, we proved almost-sure termination of the Fast Loaded Dice Roller, whose rejection-and-restart structure is captured by a single compact iteration-level argument: the supermartingale bounds the depth of the underlying decision tree, while the variant tracks the distance towards a non-rejecting leaf and resets, but does not grow unboundedly, on rejection. Combined with the existing partial-correctness result for such samplers [41], our AST proofs complete the total-correctness argument for both FDR and FLDR. In particular, the FLDR proof closes the formal termination gap left by the original algorithm presentation.

Limitations. The rule as presented has two main restrictions. First, it applies to a single while-loop with a *loop-free* body: the one-iteration distribution-transformer step that the rule reasons about is only well defined when the body itself contains no further loops. Nested loops can in principle be handled by applying the rule to the innermost loop first and treating its established AST as a black box when analyzing the enclosing loop, but we do not formalize or evaluate this modular composition here. Second, the underlying programming language does not include *nondeterministic* (demonic or angelic) choice: every probabilistic branch has a fixed, syntactically given probability, and there is no construct for resolving nondeterminism via a scheduler. This mirrors the pCFG rule’s treatment of nondeterminism-free models and keeps the connection between program syntax and one-step distribution-transformer semantics direct, but it excludes programs whose termination behavior depends on an adversarial or best-case choice of successor. As with the pCFG rule of [28], we also restrict attention to programs over rational-valued variables, which avoids measure-theoretic subtleties but excludes continuous sampling primitives.

Future work. Lifting the loop-free-body restriction to a fully compositional treatment of nested and sequentially composed loops is the most immediate extension, and would let the rule be applied automatically along a program’s syntax tree rather than loop by loop. Incorporating demonic or angelic nondeterminism, following strategy-based semantics for probabilistic programs with nondeterministic choice [10], would connect the rule to settings where a scheduler resolves nondeterministic branching, as already supported by the pCFG rule of Majumdar and Sathiyararayanan [28]. It would also be worthwhile to study the precise relationship between our rule and the program-level rule of McIver et al. [30] in more depth, in particular whether one can characterize classes of programs for which one of the two styles of certificate is systematically easier to construct than the other. Finally, the relative-completeness results now available for both pCFG-level and program-level AST rules [28, 30] suggest a concrete target for automation: since our rule decouples the supermartingale and variant obligations, each may be amenable to separate, specialized synthesis procedures, offering a path towards (semi-)automated AST proofs for sampling algorithms such as FDR and FLDR.

A Translation and AST Preservation

This appendix records the correspondence used by the metatheory. The structural translation $\mathcal{T}$ introduces entry and exit locations for every command. Assignment edges update the valuation, guarded edges encode conditionals and loop tests, and a probabilistic-choice location has outgoing

probabilities p and $1 - p$. Sequential composition identifies the first command's exit with the second command's entry.

LEMMA A.1 (LOOP-FREE CORRESPONDENCE). *For loop-free B and states σ, σ' ,*

$$\text{sp}\llbracket B \rrbracket(\delta_\sigma)(\sigma') = \Pr(\text{Paths}_{\mathcal{T}[B]}((\ell_{\text{in}}, \sigma), (\ell_{\text{out}}, \sigma'))).$$

PROOF. By structural induction on B . Skip and assignment have one path. Sequential composition uses convolution over intermediate states. Conditional paths are partitioned by the guard. Probabilistic choice multiplies branch-path probabilities by p and $1 - p$, matching linearity of the post-transformer. $\square$

For a loop, the j th Kleene approximant of its characteristic functional equals the probability distribution of pCFG runs that exit within at most j visits to the guard. Taking suprema gives equality between post-distribution weight and pCFG reachability probability. Therefore $|\text{sp}\llbracket C \rrbracket(\delta_\sigma)| = 1$ iff the translated graph reaches its exit almost surely from $(\ell_{\text{in}}, \sigma)$. Linearity extends the equivalence to arbitrary initial subdistributions.

B Detailed One-Iteration Calculations

FDR.. Starting in σ , doubling v and sampling the bit produces the two Dirac states $\sigma[v/2v, c/2c]$ and $\sigma[v/2v, c/2c+1]$, each of mass $1/2$. Filtering by $c \geq n$ and applying both subtraction assignments yields Equation (2). On the left-bit outcome, subtraction of n from both variables preserves their difference, hence in all enabled cases

$$(v' - c') = 2(v - c).$$

This is the algebraic core of the FDR variant argument.

FLDR.. After incrementing c , the fair choice reaches indices $2d$ and $2d + 1$. A rejecting child maps to $(c, d) = (0, 0)$; a non-rejecting child retains depth $c + 1$. Since preprocessing never places the unique rejecting outcome in both children of one internal node, the total mass of non-reset successors is at least $1/2$. Every such successor reduces $k - c$ by one.

C Proof-Obligation Comparison

The comparison below counts categories of obligations rather than lines of prose.

	pCFG level	program level
Invariant	one predicate per location	one distributional invariant
Supermartingale	edge/choice obligations	one expected body-step inequality
Variant	edge-wise strict/probabilistic descent	one post-distribution inequality
Control-flow constants	scaling and location offsets	none
Reuse from partial correctness	requires projection/lifting	direct

For FDR, both columns are feasible and prove the same AST fact. This makes FDR a controlled comparison of proof style. For FLDR, rejection resets create additional pCFG locations and certificate cases, while the program proof depends only on the probability mass of non-reset outcomes.

D Artifact and Proof-Development Notes

The current manuscript is a venue-independent draft. Before submission, the following items should be completed: (i) mechanize or fully expand the translation-correctness proof; (ii) state the preprocessing properties of FLDR as explicit lemmas and connect them to its reference implementation; (iii) decide whether the generalized ε_r version belongs in the main theorem; and (iv) evaluate

proof size using a stable metric, for example number of certificate cases and generated verification conditions. The anonymous author block and review options should be replaced only after a venue has been selected.

References

- [1] Rajab Aghamov, Christel Baier, Toghrol Karimov, Joris Nieuwveld, Joël Ouaknine, Jakob Piribauer, and Mihir Vahanwala. 2024. Model Checking Markov Chains as Distribution Transformers. *CoRR* abs/2406.15087 (2024). arXiv:2406.15087 doi:10.48550/ARXIV.2406.15087
- [2] Rajab Aghamov, Christel Baier, Toghrol Karimov, Joris Nieuwveld, Joël Ouaknine, Jakob Piribauer, and Mihir Vahanwala. 2024. Model Checking Markov Chains as Distribution Transformers. In *Principles of Verification (2) (Lecture Notes in Computer Science, Vol. 15261)*. Springer, 293–313.
- [3] S. Akshay, Krishnendu Chatterjee, Tobias Meggendorfer, and Dorde Zikelic. 2023. MDPs as Distribution Transformers: Affine Invariant Synthesis for Safety Objectives. In *Computer Aided Verification, Proceedings, Part III (Lecture Notes in Computer Science, Vol. 13966)*. Springer, 86–112. doi:10.1007/978-3-031-37709-9_5
- [4] S. Akshay, Krishnendu Chatterjee, Tobias Meggendorfer, and Dorde Zikelic. 2024. Certified Policy Verification and Synthesis for MDPs under Distributional Reach-Avoidance Properties. In *IJCAI*. ijcai.org, 3–12.
- [5] S. Akshay, Blaise Genest, and Nikhil Vyas. 2018. Distribution-based objectives for Markov Decision Processes. In *LICS*. ACM, 36–45.
- [6] Alexander Bagnall, Gordon Stewart, and Anindya Banerjee. 2023. Formally Verified Samplers from Probabilistic Programs with Loops and Conditioning. *Proc. ACM Program. Lang.* 7, PLDI (2023), 1–24. doi:10.1145/3591220
- [7] Alexander Bagnall, Gordon Stewart, and Anindya Banerjee. 2023. Formally Verified Samplers from Probabilistic Programs with Loops and Conditioning. *Proc. ACM Program. Lang.* 7, PLDI, Article 106 (June 2023), 24 pages. doi:10.1145/3591220
- [8] Christel Baier and Joost-Pieter Katoen. 2008. *Principles of Model Checking*. MIT Press.
- [9] Gilles Barthe, Thomas Espitau, Marco Gaboardi, Benjamin Grégoire, Justin Hsu, and Pierre-Yves Strub. 2018. An Assertion-Based Program Logic for Probabilistic Programs. In *Programming Languages and Systems (Lecture Notes in Computer Science, Vol. 10801)*. Springer, 117–144. doi:10.1007/978-3-319-89884-1_5
- [10] Kevin Batz, Tom Jannik Biskup, Joost-Pieter Katoen, and Tobias Winkler. 2024. Programmatic Strategy Synthesis: Resolving Nondeterminism in Probabilistic Programs. *Proc. ACM Program. Lang.* 8, POPL (2024), 2792–2820. doi:10.1145/3632935
- [11] Raven Beutner, C.-H. Luke Ong, and Fabian Zaiser. 2022. Guaranteed bounds for posterior inference in universal probabilistic programming. In *PLDI*. ACM, 536–551. doi:10.1145/3519939.3523721
- [12] Mingshuai Chen, Joost-Pieter Katoen, Lutz Klinkenberg, and Tobias Winkler. 2022. Does a Program Yield the Right Distribution? - Verifying Probabilistic Programs via Generating Functions. In *CAV (1) (Lecture Notes in Computer Science, Vol. 13371)*. Springer, 79–101.
- [13] Jerry den Hartog and Erik P. de Vink. 2002. Verifying Probabilistic Programs Using a Hoare Like Logic. *Int. J. Found. Comput. Sci.* 13, 3 (2002), 315–340. doi:10.1142/S012905410200114X
- [14] Edsger W. Dijkstra. 1975. Guarded Commands, Nondeterminacy and Formal Derivation of Programs. *Commun. ACM* 18, 8 (1975), 453–457. doi:10.1145/360933.360975
- [15] Edsger W. Dijkstra and Carel S. Scholten. 1990. *Predicate Calculus and Program Semantics*. Springer. doi:10.1007/978-1-4612-3228-5
- [16] Timon Gehr, Sasa Misailovic, and Martin T. Vechev. 2016. PSI: Exact Symbolic Inference for Probabilistic Programs. In *CAV (1) (Lecture Notes in Computer Science, Vol. 9779)*. Springer, 62–83. doi:10.1007/978-3-319-41528-4_4
- [17] Timon Gehr, Samuel Steffen, and Martin T. Vechev. 2020. λ PSI: exact inference for higher-order probabilistic programs. In *PLDI*. ACM, 883–897. doi:10.1145/3385412.3386006
- [18] Christoph Haase et al. 2026. Generating Functions Meet Occupation Invariants: Verifying the Fast Dice Roller. Preprint.
- [19] Christian Hensel, Sebastian Junges, Joost-Pieter Katoen, Tim Quatmann, and Matthias Volk. 2022. The probabilistic model checker Storm. *Int. J. Softw. Tools Technol. Transf.* 24, 4 (2022), 589–610.
- [20] Steven Holtzen, Van den Broeck, and Todd D. Millstein. 2020. Scaling exact inference for discrete probabilistic programs. *Proc. ACM Program. Lang.* 4, OOPSLA (2020), 140:1–140:31. doi:10.1145/3428208
- [21] Sebastian Junges, Joost-Pieter Katoen, Guillermo A. Pérez, and Tobias Winkler. 2021. The complexity of reachability in parametric Markov decision processes. *J. Comput. Syst. Sci.* 119 (2021), 183–210.
- [22] Benjamin Lucien Kaminski. 2019. *Advanced weakest precondition calculi for probabilistic programs*. Ph.D. Dissertation. RWTH Aachen University, Germany. <http://publications.rwth-aachen.de/record/755408>
- [23] Lutz Klinkenberg, Kevin Batz, Benjamin Lucien Kaminski, Joost-Pieter Katoen, Joshua Moerman, and Tobias Winkler. 2020. Generating Functions for Probabilistic Programs. In *Logic-Based Program Synthesis and Transformation (Lecture*

- Notes in Computer Science, Vol. 12561). Springer, 231–248. doi:10.1007/978-3-030-68446-4_12
- [24] Dexter Kozen. 1979. Semantics of Probabilistic Programs. In *20th Annual Symposium on Foundations of Computer Science*. IEEE Computer Society, 101–114. doi:10.1109/SFCS.1979.38
- [25] Dexter Kozen. 1983. A Probabilistic PDL. In *Proceedings of the 15th Annual ACM Symposium on Theory of Computing*. ACM, 291–297. doi:10.1145/800061.808758
- [26] John M. Li, Amal Ahmed, and Steven Holtzen. 2023. Lilac: A Modal Separation Logic for Conditional Probability. *Proc. ACM Program. Lang.* 7, PLDI (2023), 148–171. doi:10.1145/3591226
- [27] Jérémie O. Lumbroso. 2013. Optimal Discrete Uniform Generation from Coin Flips, and Applications. *CoRR* abs/1304.1916 (2013). arXiv:1304.1916 <http://arxiv.org/abs/1304.1916>
- [28] Rupak Majumdar and V. R. Sathiyarayanan. 2025. Sound and Complete Proof Rules for Probabilistic Termination. *Proc. ACM Program. Lang.* 9, POPL (2025), 1871–1902.
- [29] Annabelle McIver and Carroll Morgan. 2005. *Abstraction, Refinement and Proof for Probabilistic Systems*. Springer. doi:10.1007/B138392
- [30] Annabelle McIver, Carroll Morgan, Benjamin Lucien Kaminski, and Joost-Pieter Katoen. 2017. A new proof rule for almost-sure termination. *Proc. ACM Program. Lang.* 2, POPL, Article 33 (Dec. 2017), 28 pages. doi:10.1145/3158121
- [31] Hannah Mertens, Joost-Pieter Katoen, Tim Quatmann, and Tobias Winkler. 2024. Accurately Computing Expected Visiting Times and Stationary Distributions in Markov Chains. In *TACAS (2) (Lecture Notes in Computer Science, Vol. 14571)*. Springer, 237–257.
- [32] Martin L. Puterman. 1994. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley.
- [33] Feras Saad, Cameron E. Freer, Martin C. Rinard, and Vikash Mansinghka. 2020. The Fast Loaded Dice Roller: A Near-Optimal Exact Sampler for Discrete Probability Distributions. In *The 23rd International Conference on Artificial Intelligence and Statistics, AISTATS 2020 (Proceedings of Machine Learning Research, Vol. 108)*. PMLR, 1036–1046. <http://proceedings.mlr.press/v108/saad20a.html>
- [34] John Von Neumann. 1951. Various techniques used in connection with random digits. *Appl. Math Ser* 12, 36–38 (1951), 5.
- [35] Di Wang. 2018. A Denotational Semantics for Nondeterminism in Probabilistic Programs. <https://api.semanticscholar.org/CorpusID:52247661>
- [36] Linpeng Zhang and Benjamin Lucien Kaminski. 2022. Quantitative strongest post: a calculus for reasoning about the flow of quantitative information. *Proc. ACM Program. Lang.* 6, OOPSLA1 (2022), 1–29. doi:10.1145/3527331
- [37] Linpeng Zhang, Noam Zilberstein, Benjamin Lucien Kaminski, and Alexandra Silva. 2024. Quantitative Weakest Hyper Pre: Unifying Correctness and Incorrectness Hyperproperties via Predicate Transformers. *CoRR* abs/2404.05097 (2024). arXiv:2404.05097 doi:10.48550/ARXIV.2404.05097
- [38] Noam Zilberstein, Derek Dreyer, and Alexandra Silva. 2025. Outcome Logic for Nondeterministic Programs. *Proc. ACM Program. Lang.* (2025).
- [39] Noam Zilberstein, Dexter Kozen, Alexandra Silva, and Joseph Tassarotti. 2025. A Demonic Outcome Logic for Randomized Nondeterminism. *Proceedings of the ACM on Programming Languages* 9, POPL (Jan. 2025), 539–568. doi:10.1145/3704855
- [40] Noam Zilberstein, Angelina Saliling, and Alexandra Silva. 2025. Demonic Outcome Logic for Randomized Nondeterminism. *Proc. ACM Program. Lang.* 9, POPL (2025). doi:10.1145/3743131
- [41] Daniel Zilken, Kevin Batz, Joost-Pieter Katoen, and Tobias Winkler. 2026. Verifying Sampling Algorithms via Distributional Invariants. arXiv:2509.06410 [cs.LO] <https://arxiv.org/abs/2509.06410>